

RoboBehaviors and Finite State Machines Project

Ishan Porwal, Ahan Trivedi

For Olin ENGR3590 Computational Introduction to Robotics

Project Overview

For this project, we built a finite state controller in ROS 2 using C++ to control a Neato robot based on LiDAR and bumper sensor data. The goal was to combine multiple simple robot behaviors into one system where the robot could react to its environment and switch between behaviors automatically. Our controller has three main states: **moving forward**, **rotating when an obstacle is detected in front of the robot**, and **stopping if one of the bump sensors is triggered**.

We also used this project to get more comfortable working with ROS 2 in C++, especially with publishers, subscribers, sensor callbacks, shared pointers, and structuring robot behavior with a finite state machine. We tested the controller on the physical Neato and used ROS logging, topic inspection, RViz, and rosbag recordings to debug the sensor data and make sure the state transitions were happening when expected.

Individual Behaviors

Move Forward

Our first behavior was moving the Neato forward in a straight line. This was the default behavior when there was no obstacle detected in front of the robot and none of the bump sensors had been triggered. We implemented this by publishing a Twist message to `/cmd_vel` with a positive linear velocity and an angular velocity of zero.

We kept the forward speed pretty low so that the robot had enough time to detect obstacles and react before getting too close to them. We also kept this behavior simple on purpose, since most of the decision-making happens in the finite state controller. The move forward behavior is mainly responsible for commanding the robot forward, while the LiDAR and bump sensor callbacks determine when it should switch to another state.

Obstacle Detection and Rotation

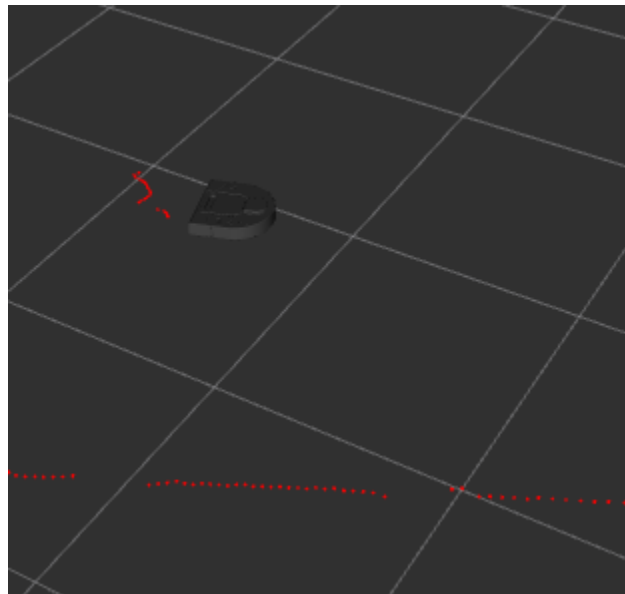
Our second behavior was detecting an obstacle in front of the Neato and rotating away from it. We subscribed to the `/scan` topic to get LiDAR measurements from the robot and only looked at the readings corresponding to the front of the Neato. Instead of relying on a single LiDAR reading, we checked multiple samples on both sides of the front of the scan and used the closest detected object.

When the closest measurement was within our chosen obstacle threshold, we treated that as an obstacle and switched to the rotate behavior. In this state, the Neato publishes zero linear velocity and a negative angular velocity to `/cmd_vel`, causing it to rotate in place. It keeps rotating until

the obstacle is no longer in the front detection region, at which point it can transition back to moving forward.

One design decision we had to make was how large the front detection region should be. At first, using too small of a section was not very reliable, so we expanded it to include more LiDAR samples. We also had to choose an obstacle distance that gave the robot enough time to react without causing it to turn when objects were still pretty far away.

We tested this behavior by having a person stand in front of the Neato to simulate an obstacle. The LiDAR detected the person, the robot transitioned into the rotate state, and it kept rotating until the person was no longer in front of it. Once the path was clear again, the robot transitioned back to moving forward.



RViz visualization of the Neato's LiDAR detecting a person standing in front of the robot during obstacle avoidance testing.

Emergency Stop

Our third behavior was an emergency stop using the Neato's physical bump sensors. We subscribed to the `/bump` topic and checked the front and side bump sensors on both sides of the robot. If any of these sensors were triggered, we set a bump detection flag that caused the finite state controller to transition into the stop state.

In the stop state, the Neato publishes zero linear and angular velocity to `/cmd_vel`, causing it to stop moving completely. We wanted the bump sensors to act as a final safety response in case the LiDAR did not detect an obstacle in time or the robot still made physical contact with something.

Once the Neato entered the stop state, we kept it stopped instead of having it automatically start moving again.

We tested this behavior by letting the robot move forward and then physically pressing one of the bump sensors. As soon as the bumper was triggered, the robot transitioned into the stop state and stopped moving.

Finite State Machine

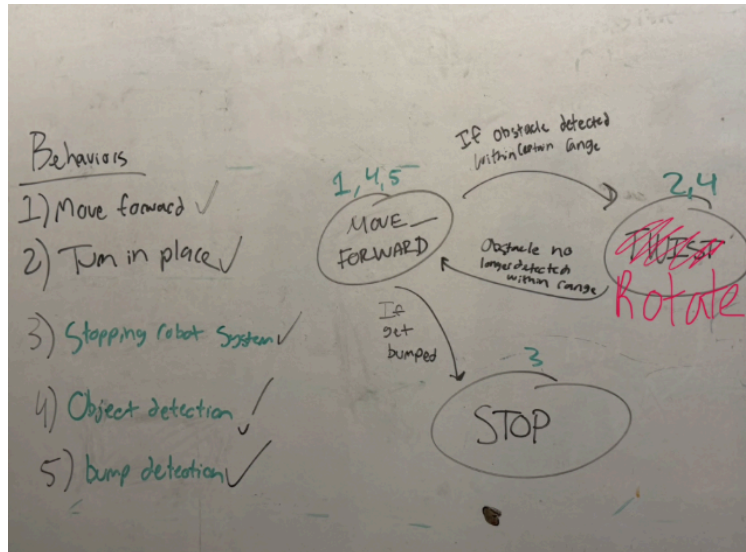
The overall goal of our finite state controller was to let the Neato move through the environment while reacting to obstacles and physical contact. The robot starts in the move forward state and continues driving straight as long as there is no obstacle detected in front of it. If the LiDAR detects an obstacle within our front detection region, the controller transitions into the rotate state. The robot then rotates in place until the obstacle is no longer in front of it, at which point it transitions back to moving forward.

The third state is stop, which is triggered if any of the Neato's bump sensors are pressed. We treated this as a higher-priority condition than the normal obstacle avoidance behavior, since it means the robot has physically contacted something. Once the controller enters the stop state, the robot remains stopped.

We combined the behaviors by keeping the sensor processing separate from the actual state logic. The LiDAR callback updates whether an obstacle is currently detected, while the bump callback updates whether a bump has occurred. The finite state controller then checks those values every 100 ms and decides which state the robot should be in. Each state calls its corresponding behavior function to publish the correct velocity command.

The main transitions in our controller were:

- Move Forward → Rotate when an obstacle is detected in the front LiDAR region
- Rotate → Move Forward when the obstacle is no longer detected in front of the robot
- Move Forward → Stop if a bump sensor is triggered
- Rotate → Stop if a bump sensor is triggered
- Stop has no outgoing transition, so the robot stays stopped once a bump occurs



State transition diagram showing how the Neato switches between moving forward, rotating around an obstacle, and stopping after a bump is detected.

Demonstration

We tested the full finite state controller on the physical Neato to make sure all three behaviors worked together as expected. During the demo, the robot started by moving forward, detected a person standing in front of it using the LiDAR, transitioned into the rotate state, and continued rotating until the path in front of it was clear. It then transitioned back to moving forward. We also tested the bump sensors during the same run, which caused the robot to transition into the stop state and remain stopped.

This video shows the complete behavior of the finite state controller running on the physical Neato:  [IMG_8920.MOV](#)

Rosbag: [Finite State Controller Demo Bag](#)

Code Structure

We structured the project around a single ROS 2 node called `FiniteStateController` (implemented in [finite_state_controller.cpp](#)). The class inherits from `rclcpp::Node` and is responsible for setting up the publishers, subscribers, timer, sensor callbacks, and finite state machine logic.

The constructor initializes the controller in the `move_forward` state and creates the ROS interfaces we need. We subscribe to `/scan` for LiDAR data and `/bump` for the Neato's bump sensors, and publish velocity commands to `/cmd_vel`. We also use a 100 ms timer to repeatedly run the main finite state controller logic.

The sensor callbacks are kept separate from the state logic. `scan_callback()` processes the LiDAR data and updates whether an obstacle is currently detected in front of the robot, while `bump_callback()` checks the bump sensors and updates whether the robot has made physical contact with something. These callbacks update internal state variables rather than directly controlling the robot.

The actual robot behaviors are split into three separate functions: `move_forward()`, `rotate()`, and `stop()`. Each function creates and publishes the appropriate `Twist` command for that behavior. The `run_loop()` function then acts as the main finite state controller by checking the current sensor states, deciding whether a transition should happen, and calling the behavior associated with the current state.

We chose this structure because it kept the sensor processing, state transition logic, and robot commands separate from each other. This made the code easier to debug and also made it easier to understand which part of the program was responsible for each behavior.

Challenges

One of the main challenges we ran into was figuring out how to correctly use the LiDAR data for obstacle detection. At first, it was not immediately obvious which parts of the `/scan` array corresponded to the front of the Neato. We had to look at how the scan was organized and test different sections before settling on using samples from both ends of the array. We also started with too small of a detection region, which made the behavior less reliable, so we increased the number of LiDAR samples we checked.

Another challenge was testing the bump behavior in simulation. We expected the simulated Neato to trigger the bump sensor when it physically ran into an obstacle, but that behavior was not working reliably for us. Instead of spending too much time trying to force the simulation to behave correctly, we moved our testing to the physical Neato. Once we tested it in person, the bump sensors worked as expected and the robot transitioned into the stop state correctly.

We also spent some time figuring out how to debug the system when we first started working with the Neato. Since there were multiple things happening at once, like sensor data, state transitions, and velocity commands, it was not always obvious where an issue was coming from. We ended up using ROS logging, `ros2 topic echo`, RViz, and rosbag recordings to check each part individually. This made it much easier to tell whether a problem was coming from the sensor data, our state logic, or the robot itself.

Future Improvements

If we had more time, we would want to expand the controller with more advanced behaviors instead of only reacting to obstacles. One example would be person following, where the Neato could use sensor data to identify and follow a person while still avoiding obstacles. We could also add behaviors like wall following or navigating toward a target, which would make the finite state controller more capable and give us more interesting transitions between behaviors.

We would also improve the stop state. For this project, we treated it almost like an emergency stop, so once a bump sensor was triggered the robot stayed stopped. That worked well for what we were trying to demonstrate, but in a more complete system we would want the robot to be able to recover, either by resetting the controller or by backing up and continuing once the obstacle was cleared.

Learning Objectives and Key Takeaways

Ishan's main learning objective for this project was to get more practice debugging a full robotics system and using simulation as part of the development process. Ahan's main learning objective was to get more comfortable structuring robotics code in C++, especially organizing a ROS node and separating sensor handling from the finite state machine logic.

One of our biggest takeaways was how useful finite state machines are for organizing robot behavior. Breaking the robot into clear states made it much easier to reason about what it should be doing and when it should transition between behaviors. We also got more comfortable with how publishers, subscribers, callbacks, and timers work together inside a ROS 2 node.

We also learned a lot about debugging robotics systems. Tools like ROS logging, `ros2 topic echo`, `RViz`, and `rosbag` recordings helped us isolate issues across the sensor data, state logic, and robot behavior instead of trying to debug everything at once.

Finally, testing on the physical Neato showed us how important real-world testing is. Some things that seemed straightforward in simulation, especially the bump sensors, behaved differently on the actual robot, which showed us how much iteration can be needed to get even simple robot behaviors working reliably.

How To Run

To run the project on a physical Neato, first make sure both the laptop and the Neato are connected to the same network and note the IP address shown on the Neato.

Clone the project repository into the `src` directory of a ROS 2 workspace:

Shell

```
cd ~/ros2_ws/src
```

Shell

```
git clone  
https://github.com/ishanporwal/comprobo-warmup-project.git
```

Then return to the workspace root, source ROS 2, build the package, and source the workspace:

Shell

```
cd ~/ros2_ws
```

Shell

```
source /opt/ros/jazzy/setup.bash
```

Shell

```
colcon build --packages-select ros_behaviors_fsm
```

Shell

```
source install/setup.bash
```

In a separate terminal, connect to the physical Neato using its IP address:

Shell

```
cd ~/ros2_ws
```

Shell

```
source /opt/ros/jazzy/setup.bash  
source install/setup.bash
```

Shell

```
ros2 launch neato_node2 bringup.py host:=<NEATO_IP>
```

Once the Neato is connected and its ROS topics are available, run the finite state controller in another terminal:

Shell

```
cd ~/ros2_ws
```

Shell

```
source /opt/ros/jazzy/setup.bash  
source install/setup.bash
```

Shell

```
ros2 run ros_behaviors_fsm finite_state_controller
```

The Neato should begin in the move forward state. If it detects an obstacle in front of it using the LiDAR, it should rotate until the obstacle is no longer in front of the robot and then continue moving forward. If one of the bump sensors is triggered, the robot should transition into the stop state and remain stopped.

We also included a rosbag recording of the finite state controller running on the physical Neato. To inspect the bag, run:

Shell

```
ros2 bag info  
src/ros_behaviors_fsm/bags/finite_state_controller_demo
```

To replay it:

Shell

```
ros2 bag play  
src/ros_behaviors_fsm/bags/finite_state_controller_demo
```

The bag contains the LiDAR scans, bump sensor data, velocity commands, odometry, and transforms from our test run. It can also be replayed while using RViz to visualize the recorded sensor data and robot movement.

For safety, the robot should be placed in an open area before starting the controller since running the node immediately begins publishing movement commands.

Team Contributions

Ishan and Ahan worked on the entire project together from start to finish. Before writing any code, we whiteboarded the finite state machine and overall code architecture together so we had a clear idea of how the behaviors, sensor callbacks, and state transitions would fit together.

For the implementation, we connected Ishan's computer to the projector and worked through the code side by side rather than splitting the project into separate pieces. We also debugged the simulation together, including the issues we had getting the bump behavior to work reliably, and then moved to the physical Neato for further testing. All of the final testing, debugging, rosbag recording, and demoing were also done together.